Conn2Conn grid estimators — the mathematics

objectives, equations, matrix dimensions, and learning rules

1 Notation and dimensions

One frozen split has $n = 683$ train, 79 val, $m = 195$ test subjects. A connectome is an upper-triangle edge vector of width p (Glasser $p = 64{,}620$; 4S456Parcels $p = 103{,}740$).

Symbol	Shape	Meaning
$\mathbf{X}$	$\mathbb{R}^{n\times p}$	train source connectome (n=683 rows)
$\mathbf{X}_\star$	$\mathbb{R}^{m\times p}$	test source connectome (m=195)
$\mathbf{Y}$	$\mathbb{R}^{n\times q}$	train target connectome ($q \approx p$)
k_s	256	source PCA rank, $k_s = \min(256, p)$
k_t	256	target PCA rank, $k_t = \min(256, q)$
ℓ	64	PLS latent components (32 on the scalar path)
$\mathbf{U}_x$	$\mathbb{R}^{p\times k_s}$	source PCA loadings (orthonormal columns)
$\mathbf{U}_y$	$\mathbb{R}^{q\times k_t}$	target PCA loadings
$\mathbf{Z} = \mathbf{X}\mathbf{U}_x$	$\mathbb{R}^{n\times k_s}$	source latent scores (train)
$\mathbf{W} = \mathbf{Y}\mathbf{U}_y$	$\mathbb{R}^{n\times k_t}$	target latent scores (train)

The key structural fact: $p, q \sim 10^5$ but $n = 683$, so the edge-space regression $\mathbf{Y} \approx \mathbf{X}\mathbf{B}$, $\mathbf{B} \in \mathbb{R}^{p\times q}$ has $\sim 10^{10}$ unknowns and is hopelessly rank-deficient. All three estimators avoid it by working in the ≤ 256-dim PCA latent space. Only the latent map $f : \mathbf{Z} \to \mathbf{W}$ differs between them.

2 Shared front-end: PCA compression (and its inverse)

Objective. PCA finds the orthonormal basis maximizing retained variance (equivalently minimizing reconstruction error). For the source, with centered $\widetilde{\mathbf{X}} = \mathbf{X} - \mathbf{1}\bar{\mathbf{x}}^\top$,

$$\mathbf{U}_x \;=\; \underset{\mathbf{U}^\top\mathbf{U}=\mathbf{I}_{k_s}}{\arg\max}\; \operatorname{tr}\!\big(\mathbf{U}^\top \widetilde{\mathbf{X}}^\top \widetilde{\mathbf{X}}\,\mathbf{U}\big) \;=\; \underset{\operatorname{rank}\le k_s}{\arg\min}\; \big\|\widetilde{\mathbf{X}} - \widetilde{\mathbf{X}}\,\mathbf{U}\mathbf{U}^\top\big\|_F^2.$$

The solution is the top-k_s eigenvectors of the $p \times p$ covariance $\mathbf{C} = \frac{1}{n-1}\widetilde{\mathbf{X}}^\top\widetilde{\mathbf{X}}$.

Naive learning. Form $\mathbf{C} \in \mathbb{R}^{p\times p}$ and eigendecompose: $O(p^2 n + p^3)$. With $p = 10^5$ this is $\sim 10^{15}$ flops and $\mathbf{C}$ alone is 40 GB — *infeasible*, and pointless because $\operatorname{rank}(\widetilde{\mathbf{X}}) \le n - 1 = 682 \ll p$.

Optimized learning (what runs). Since $n \ll p$, use the economy SVD $\widetilde{\mathbf{X}} = \mathbf{A}\,\mathbf{\Sigma}\,\mathbf{B}^\top$ with $\mathbf{A} \in \mathbb{R}^{n\times r}$, $\mathbf{\Sigma} \in \mathbb{R}^{r\times r}$, $\mathbf{B} \in \mathbb{R}^{p\times r}$, $r \le n-1$. Then $\mathbf{U}_x = \mathbf{B}_{:,1:k_s}$ and the scores are read off directly, $\mathbf{Z} = \widetilde{\mathbf{X}}\mathbf{U}_x = \mathbf{A}\mathbf{\Sigma}$ (truncated). The cheap route is the *Gram trick*: eigendecompose the $n \times n$ matrix $\widetilde{\mathbf{X}}\widetilde{\mathbf{X}}^\top$ (683×683) to get $\mathbf{A}, \mathbf{\Sigma}$, then $\mathbf{B} = \widetilde{\mathbf{X}}^\top \mathbf{A}\mathbf{\Sigma}^{-1}$. Cost $O(n^2 p)$ — linear in p. `sklearn.decomposition.PCA(random_state=0)` uses truncated/randomized SVD, the same idea.

Inverse-PCA (P4). Predictions live in latent space and are pushed back to edges by the (orthonormal) adjoint:

$$\widehat{\mathbf{Y}}_\star \;=\; \widehat{\mathbf{W}}_\star\,\mathbf{U}_y^\top + \mathbf{1}\,\bar{\mathbf{y}}^\top, \qquad \widehat{\mathbf{W}}_\star \in \mathbb{R}^{m\times k_t},\;\; \mathbf{U}_y^\top \in \mathbb{R}^{k_t\times q}.$$

This is a fixed linear decoder; no learning. Everything below produces $\widehat{\mathbf{W}}_\star$.

3 Estimator 1 — PCA–PLS

Objective. Partial Least Squares extracts $\ell = 64$ paired latent directions $(\mathbf{w}_a, \mathbf{c}_a)$ that maximize the *covariance* of the projected source and target scores:

$$(\mathbf{w}_a, \mathbf{c}_a) \;=\; \underset{\|\mathbf{w}\|=\|\mathbf{c}\|=1}{\arg\max}\; \mathrm{Cov}\big(\mathbf{Z}_{a-1}\mathbf{w},\; \mathbf{W}_{a-1}\mathbf{c}\big) \;=\; \underset{\|\mathbf{w}\|=\|\mathbf{c}\|=1}{\arg\max}\; \mathbf{w}^\top\,\mathbf{Z}_{a-1}^\top \mathbf{W}_{a-1}\,\mathbf{c},$$

subject to orthogonality of successive source scores $\mathbf{t}_a = \mathbf{Z}_{a-1}\mathbf{w}_a$. Unlike PCA (variance of $\mathbf{Z}$ alone) or OLS (correlation), PLS targets shared *cross-modal* covariance — the right objective when both blocks are high-dim and collinear.

Structure. For component a, the optimum $(\mathbf{w}_a, \mathbf{c}_a)$ is the leading singular vector pair of the $k_s \times k_t$ cross-covariance $\mathbf{M}_{a-1} = \mathbf{Z}_{a-1}^\top \mathbf{W}_{a-1}$. The blocks are then *deflated* (rank-1 removed) and the process repeats. After ℓ components the map collapses to a single **linear, rank-$\le \ell$** operator

$$\boxed{\widehat{\mathbf{W}}_\star = \mathbf{Z}_\star\,\mathbf{B}_{\mathrm{PLS}}}, \qquad \mathbf{B}_{\mathrm{PLS}} = \mathbf{R}\,\mathbf{Q}^\top \in \mathbb{R}^{k_s\times k_t}, \quad \mathbf{R} = \mathbf{W}^*\big(\mathbf{P}^\top\mathbf{W}^*\big)^{-1},$$

where $\mathbf{W}^* = [\mathbf{w}_1 \cdots \mathbf{w}_\ell] \in \mathbb{R}^{k_s\times\ell}$ (x-weights), $\mathbf{P} \in \mathbb{R}^{k_s\times\ell}$ (x-loadings), $\mathbf{Q} \in \mathbb{R}^{k_t\times\ell}$ (y-loadings). So PLS = a deliberately **low-rank ridge-like linear regression**; $\ell = 64 \ll k_s = 256$ is the regularizer. (`scale=True` standardizes every column of $\mathbf{Z}, \mathbf{W}$ first.)

Naive learning. Per component, full SVD of $\mathbf{M}_{a-1} \in \mathbb{R}^{256\times256}$: $O(k_s k_t \min(k_s, k_t))$ per step $\times\,\ell$ — wasteful, since only the *top* singular pair is needed.

Optimized learning (NIPALS, what `sklearn` runs). Get just the leading singular vector by power iteration, then deflate. This is the algorithm:

Algorithm 1 NIPALS for PLS2 (mode A) — ℓ components

$\mathbf{Z}_0 \leftarrow \mathbf{Z},\ \mathbf{W}_0 \leftarrow \mathbf{W}$ ▷ column-scaled
for $a = 1, \dots, \ell$ **do**
  init $\mathbf{c}$;
  repeat
    $\mathbf{w} \leftarrow \mathbf{Z}_{a-1}^\top \mathbf{W}_{a-1}\mathbf{c},\ \mathbf{w} \leftarrow \mathbf{w}/\|\mathbf{w}\|$ ▷ x-weights
    $\mathbf{t} \leftarrow \mathbf{Z}_{a-1}\mathbf{w}$ ▷ x-scores, $\in \mathbb{R}^n$
    $\mathbf{c} \leftarrow \mathbf{W}_{a-1}^\top \mathbf{t}/(\mathbf{t}^\top\mathbf{t}),\ \mathbf{c} \leftarrow \mathbf{c}/\|\mathbf{c}\|$ ▷ y-weights
  until $\mathbf{w}$ converged ▷ = power iteration on $\mathbf{M}_{a-1}\mathbf{M}_{a-1}^\top$
  $\mathbf{p} \leftarrow \mathbf{Z}_{a-1}^\top \mathbf{t}/(\mathbf{t}^\top\mathbf{t});\quad \mathbf{q} \leftarrow \mathbf{W}_{a-1}^\top \mathbf{t}/(\mathbf{t}^\top\mathbf{t})$ ▷ loadings
  $\mathbf{Z}_a \leftarrow \mathbf{Z}_{a-1} - \mathbf{t}\mathbf{p}^\top;\quad \mathbf{W}_a \leftarrow \mathbf{W}_{a-1} - \mathbf{t}\mathbf{q}^\top$ ▷ deflate
end for
assemble $\mathbf{B}_{\mathrm{PLS}} = \mathbf{R}\mathbf{Q}^\top$

Cost $\approx O(\ell \cdot nk_s)$ — linear in everything, closed-form and deterministic.

4 Estimator 2 — PCA–BayesianRidge (per target component)

The $k_t = 256$ target latents are fit *independently*: one Bayesian linear regression per column $\mathbf{y}_k = \mathbf{W}_{:,k} \in \mathbb{R}^n$, $k = 1, \dots, 256$.

Probabilistic model / loss. For target column k,

$$\mathbf{y}_k = \mathbf{Z}\,\boldsymbol{\beta}_k + \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \alpha^{-1}\mathbf{I}_n), \quad \boldsymbol{\beta}_k \sim \mathcal{N}(\mathbf{0}, \lambda^{-1}\mathbf{I}_{k_s}).$$

The negative log-posterior is a **ridge loss with a learned penalty** $\rho = \lambda/\alpha$:

$$\boldsymbol{\beta}_k^{\mathrm{MAP}} = \arg\min_{\boldsymbol{\beta}} \ \tfrac{\alpha}{2}\|\mathbf{y}_k - \mathbf{Z}\boldsymbol{\beta}\|^2 + \tfrac{\lambda}{2}\|\boldsymbol{\beta}\|^2.$$

Unlike plain ridge, the noise precision α and weight precision λ are not fixed — they are estimated from the data by maximizing the marginal likelihood (evidence), giving each of the 256 modes its own automatically-tuned shrinkage (no cross-validation).

Structure (posterior). Gaussian, closed-form:

$$\boldsymbol{\Sigma}_k = \left(\lambda\mathbf{I}_{k_s} + \alpha\,\mathbf{Z}^\top\mathbf{Z}\right)^{-1}, \qquad \mathbf{m}_k = \alpha\,\boldsymbol{\Sigma}_k\,\mathbf{Z}^\top\mathbf{y}_k, \qquad \widehat{\mathbf{W}}_{\star,k} = \mathbf{Z}_\star\,\mathbf{m}_k.$$

Prediction is again **linear** per component; stacking gives $\widehat{\mathbf{W}}_\star = \mathbf{Z}_\star\mathbf{M}$, $\mathbf{M} = [\mathbf{m}_1 \cdots \mathbf{m}_{256}] \in \mathbb{R}^{k_s \times k_t}$.

Naive learning. Alternate: (i) solve the $k_s \times k_s$ system for $\mathbf{m}_k$; (ii) update α, λ from the evidence. Each iteration re-inverts/solves $(\lambda\mathbf{I} + \alpha\mathbf{Z}^\top\mathbf{Z})$ at $O(k_s^3) = 256^3$, repeated up to `max_iter` (300 recon / 500 scalar) *per component* $\times 256$ components — the 256^3 inside the loop is the waste.

Optimized learning (what `sklearn` runs). Take the SVD $\mathbf{Z} = \mathbf{A}\,\mathrm{diag}(\sigma_i)\,\mathbf{G}^\top$ *once*. In its eigenbasis every quantity is diagonal, so each evidence update is $O(k_s)$ instead of $O(k_s^3)$:

$$\text{effective d.o.f.} \quad \gamma = \sum_i \frac{\sigma_i^2}{\sigma_i^2 + \lambda/\alpha},$$

$$\lambda \leftarrow \frac{\gamma}{\mathbf{m}_k^\top \mathbf{m}_k}, \qquad \alpha \leftarrow \frac{n - \gamma}{\|\mathbf{y}_k - \mathbf{Z}\mathbf{m}_k\|^2},$$

$$\mathbf{m}_k = \mathbf{G}\,\mathrm{diag}\Big(\tfrac{\sigma_i}{\sigma_i^2 + \lambda/\alpha}\Big)\mathbf{A}^\top \mathbf{y}_k.$$

Iterate $\alpha, \lambda, \mathbf{m}_k$ to convergence. One SVD amortizes all 256 fits; deterministic. (**Note — scalar path:** for cognition/sex/age the $\mathbf{Z}$ are cast to `float64` *before* the PCA, because the collinear `bv+demo` one-hots make the float32 SVD rank-deficient and drop demographic directions split-dependently.)

5 Estimator 3 — PCA–KernelRidge (RBF), a 3×3 sweep

The one nonlinear estimator. Source latents are first standardized, $\mathbf{Z} \to \mathbf{Z}$ with zero mean / unit variance per column (`StandardScaler`).

Feature map / kernel. An RBF kernel implicitly lifts each $\mathbf{z} \in \mathbb{R}^{k_s}$ into an infinite-dim feature space $\phi(\mathbf{z})$ with inner product

$$\kappa(\mathbf{z}_i, \mathbf{z}_j) = \exp\big(-\gamma\,\|\mathbf{z}_i - \mathbf{z}_j\|^2\big), \qquad \mathbf{K} \in \mathbb{R}^{n \times n}, \quad \mathbf{K}_{ij} = \kappa(\mathbf{z}_i, \mathbf{z}_j).$$

$\mathbf{K}$ is 683×683 (cf. the 256×256 latent operators of the linear models).

Objective (RKHS). For the multi-output target $\mathbf{W}$,

$$\widehat{f} = \arg\min_{f \in \mathcal{H}} \sum_{i=1}^{n} \|\mathbf{w}_i - f(\mathbf{z}_i)\|^2 + \alpha\,\|f\|_{\mathcal{H}}^2.$$

Structure (representer theorem). The minimizer is a kernel expansion on the training points, $f(\mathbf{z}) = \sum_i \kappa(\mathbf{z}, \mathbf{z}_i)\,\boldsymbol{\beta}_i$, with dual coefficients

$$\boxed{\boldsymbol{\beta} = (\mathbf{K} + \alpha \mathbf{I}_n)^{-1}\mathbf{W}} \in \mathbb{R}^{n \times k_t}, \qquad \widehat{\mathbf{W}}_\star = \mathbf{K}_\star\,\boldsymbol{\beta},$$

where the test–train kernel $\mathbf{K}_\star \in \mathbb{R}^{m \times n}$ has $(\mathbf{K}_\star)_{ij} = \exp(-\gamma\|\mathbf{z}_i^\star - \mathbf{z}_j\|^2)$. The prediction is **linear in $\mathbf{K}_\star$ but nonlinear in $\mathbf{z}$** — that is the whole point of including it (a genuine nonlinear probe).

Hyperparameters (the 3×3 grid).

- **Bandwidth** $\gamma = \gamma_{\text{med}} \cdot m_\gamma$, with the median heuristic $\gamma_{\text{med}} = \dfrac{1}{2 \cdot \text{median}_{i<j}\|\mathbf{z}_i - \mathbf{z}_j\|^2}$ (computed on a deterministic 300-point subsample), $m_\gamma \in \{0.5, 1, 2\}$.
- **Ridge** $\alpha \in \{0.1, 1, 10\}$.

$\Rightarrow$ 9 variants. Empirically `demeaned_pearson` varies by only ~ 0.004 across the grid — the surface is flat (the null result that nonlinearity/HP-tuning buys nothing here).

Naive vs. optimized learning. The solve is a single linear system $(\mathbf{K} + \alpha\mathbf{I})\boldsymbol{\beta} = \mathbf{W}$. Naively, invert $\mathbf{K}+\alpha\mathbf{I}$ explicitly ($O(n^3)$ then a matmul). Optimized (`sklearn.kernel_ridge`): one Cholesky factorization $\mathbf{K} + \alpha\mathbf{I} = \mathbf{L}\mathbf{L}^\top$ ($O(n^3) = 683^3$, done *once*), then all $k_t = 256$ right-hand sides solved by back-substitution ($O(n^2 k_t)$). At $n = 683$ the exact solve is cheap, so no Nyström/conjugate-gradient approximation is used — the kernel is dense and exact.

6 One-line contrast

Model	Objective	Solution	Learns by	Map $\mathbf{Z} \to \mathbf{W}$
PCA–PLS	max cross-cov.	$\mathbf{B} = \mathbf{R}\mathbf{Q}^\top$, rank $\leq \ell$	NIPALS power iter. + deflation	linear (low-rank)
PCA–BR	ridge w/ learned ρ	$\mathbf{m}_k = \alpha\boldsymbol{\Sigma}_k\mathbf{Z}^\top\mathbf{y}_k$	evidence max. (SVD once)	linear
PCA–KRR	RKHS ridge	$\boldsymbol{\beta} = (\mathbf{K}+\alpha\mathbf{I})^{-1}\mathbf{W}$	one Cholesky solve	**nonlinear** in $\mathbf{z}$

All three are deterministic and closed-form (or fixed-point); the only randomness is the seeded truncated-SVD inside PCA, fixed by `random_state=0`. Two of three produce a linear latent map and differ only in *how the* 256×256 *coefficient matrix is regularized* (low-rank covariance vs. per-mode evidence ridge); the third lifts to an RKHS and is the lone nonlinear test — which comes back null.